

## **REVIEW 1 REPORT**

BCSE307P Compiler Design Lab

# **Interactive Compiler Interface**

A Two-Way Developer–Compiler Interaction Model

### **Submitted By**

Prashidika Upadhyay (24BCB0167)

### **Faculty In-Charge**

Dr. Ranjithkumar S

# 1. Project Title

Formal title: Interactive Compiler Interface: A Two-Way Developer–Compiler Interaction Model.

The title represents the core of the project: it is about changing the way a developer communicates with the compiler such that the interaction is no longer merely a one-way message telling the developer what went wrong but a small, structured conversation on the compiler's current understanding. The term "compiler" in the title is used when talking about a dialogue between two active participants in which the user and the compiler interact to find a solution rather than as a passive checker which would only give pass-or-fail results.

## 2. Abstract

In traditional methods, a compiler works in a very straightforward step-by-step order. First, a developer sends some code that he or she is writing. The compiler then parses it and gives out a message, typically a list of diagnostics, or sometimes, no message at all if everything went fine. After seeing the message, the developer alone has to decipher the diagnostic and then figure out the best possible way to fix the issue. But, in most cases after that message is shown, even the diagnostic message gets rid of, but still, all the compiler's extra analysis context, which is rich beyond the message, is not exposed to the developer as a persistent, user-readable interaction state via the conventional diagnostic interface. The developer will then have to do another problem-solving cycle which involves analyzing the code and finding where exactly the problem lies, rather than continuing the same line of inquiry as supported by the compiler. An Interactive Compiler Interface, that means, proposes a different way of this flow. The main idea of the tool is the one that keeps the rich structure of a source analysis by the compiler alive as a sort of 'persistent' analysis state which is, so to say, a session of a fixed length and the user can interact with that state through a set of small pre-defined commands which will give the user an opportunity to continue and elaborate on their questions rather than only being restricted to the initial error messages.

Overall, the project is centered around a small, restricted imperative programming language and a regular compiler-like pipeline that includes the stages of lexical analysis, syntax analysis, construction of abstract grammar tree, semantic analysis, symbol table management, a simple intermediate representation, and a tree interpreter. Each of these phases generates information which a compiler normally does not show you. The Interactive Compiler Interface shows that information to the user through a diagnostics model based on hierarchical data instead of plain text, and surrounds the pipeline with both a session manager and an interaction manager, so commands like why explain see symbols, suggest fixes, and apply will be resolved against the current state of compilation. Together, these make up the complete set of features that are considered and planned for the Interactive Compiler Interface system (the Review 1 prototype being its first, deliberately bounded vertical slice).

The scope of the experiment is intentionally narrow. Its repairs are limited to a small set of explicitly defined, deterministic transformations, and the boundaries of the project as a whole are stated in Section 6.3. The contribution under investigation is the interaction model itself as the idea that a compiler's active analysis context can remain addressable during a session and the small compiler pipeline exists to allow that model to be implemented and observed directly rather than only described.

This Phase 1 report documents the problem, motivation, objectives, scope, relevant background, compiler-design concepts, proposed methodology, system architecture, technology stack, and an initial Review 1 prototype. The Review 1 prototype is defined as one complete, minimal vertical slice of the interaction model: it detects a missing statement terminator, converts the resulting parser error into a structured diagnostic, retains that diagnostic as active session state, allows the developer to query it through the explain and suggest fixes commands, applies a deterministic repair through the apply 1 command, and genuinely recompiles the corrected source to verify the repair. This vertical slice is intended to demonstrate the feasibility of the central interaction idea end to end; it does not represent completion of the full planned compiler pipeline described above, which remains the subject of subsequent implementation phases. The implementation status of each prototype component, as verified against working code, is recorded separately in Section 12.4.

## **3. Problem Statement**

### **3.1 Existing Interaction Pattern**

A compiler translates source code through consecutive passes of analysis, and in each analysis phase it keeps data which can potentially help us, e.g. the type of the most recently read token, the precise location where a grammar rule mismatched, the type of the expression deduced automatically or the symbol table contents as they are visible in the scope where we are at the moment. But in normal circumstances, the very little amount of information provided by the compiler is not directly accessible to the developer. Usually the message is condensed to a single diagnostic error message which is shown once and the context of the message which includes the more in-depth analysis is seldom presented as a state which is persistent and which could be queried later. Developer gets that one message, he has to figure out what the error is without a guidance, choose a right fix, modify the file, and compile the code. The same analysis message interpretation, edit cycle is done over and over, if the correction turns out wrong or not enough.

### **3.2 Problem Identified**

This project addresses the narrowness of that interaction model. A diagnostic message is, in effect, the compiler ending the conversation at the first sentence. The compiler has already done the analytical work needed to say more, it knows where the failure occurred, what the grammar or semantic rule expected, and in many cases what a minimal correction would look like but the conventional diagnostic interface does not present this as a state the developer can continue to query. Once the message is printed, the exchange is over as far as that interface is concerned, and any further exploration of the problem must be reconstructed by the developer rather than continued with the compiler.

### **3.3 Significance of the Problem**

This pattern places an avoidable burden on the developer, particularly when a diagnostic is terse or when the underlying rule that was violated is not immediately obvious from the message alone. It also leaves unused analytical work that the compiler has already performed: symbol-table entries, partially built AST nodes, and precise source locations are all computed, but are not made addressable for the few seconds during which they would still be useful to the developer. Examining whether this information can instead be kept addressable is a reasonable and technically grounded question for a compiler-design project, since

it engages directly with how diagnostic and analysis data is represented internally rather than only with how it is displayed.

### **3.4 Research Question**

The project investigates a single, clearly bounded question: what if the compiler's current analysis context could remain addressable for an ongoing interaction with the developer, instead of the exchange ending once a diagnostic message is printed?

### **3.5 Proposed Resolution (High Level)**

Interactive Compiler Interface proposes a controlled answer to this question rather than a general one. A small interaction layer is placed around an otherwise conventional compiler pipeline. When a compiler stage produces a diagnostic, that diagnostic is represented as structured data and held, together with the relevant compiler state, by a session manager. The developer can then issue a small, fixed vocabulary of commands against this retained state to request an explanation, inspect symbol information, list candidate repairs, or apply one of those repairs, after which the interaction manager resolves the command, and, where source code is changed, the pipeline is invoked again. The compiler pipeline itself is not altered by this arrangement; the interaction layer only exposes and acts upon state the pipeline already produces. The Review 1 prototype in Section 12 is defined to realise this resolution for one concrete diagnostic, as a first, verifiable instance of the general mechanism described here.

## **4. Motivation**

### **4.1 Why the Idea Arose**

The motivation for Interactive Compiler Interface comes from the gap between how much structured information a compiler builds while analysing a program and how little of that information ordinarily reaches the developer. Each compilation phase produces structured internal data, tokens, an AST, a symbol table, type information, yet the developer typically receives only the final diagnostic string and is left to reconstruct the reasoning behind it.

### **4.2 Compiler-Internal Information vs. Developer-Facing Interaction**

There is a clear asymmetry between what a compiler knows internally at the moment of failure and what it communicates externally. A type-checking pass, for instance, knows the inferred type of an expression, the expected type at that position, and the specific rule that was violated; the developer usually sees only a one-line summary of the mismatch. Interactive Compiler Interface treats this asymmetry as the object of study: it does not add new analysis capability to the compiler, but changes what happens to the analysis results that already exist at the moment of an error.

### **4.3 Interaction as an Exchange**

The project's working hypothesis is that a diagnostic message can be the opening line of a short exchange rather than its conclusion. If the compiler reports “expected ‘;’”, a developer should reasonably be able to ask the system to explain that message, request a list of supported fixes, and choose one, with the compiler

answering each request because the relevant diagnostic and compiler state have deliberately been kept in memory for that purpose.

#### **4.4 Design Philosophy**

Interactive Compiler Interface does not attempt to modify how a production compiler behaves, nor does it try to demonstrate the idea only through static examples or mock-ups. Instead, it uses a small, controlled compiler as a safe environment in which the interaction model can be built and run for real. Keeping the underlying language and pipeline simple is a deliberate design choice: it allows the interaction mechanism, session state, structured diagnostics, bounded repairs, to be implemented completely within the time available for an individual semester project, and to be observed working end to end rather than only argued for. The Review 1 prototype applies this philosophy at its smallest possible scale: a single diagnostic, to be demonstrated on running code rather than only described.

#### **4.5 A Controlled “What-If” Experiment**

Framed this way, the project poses a specific and answerable what-if question: if a compiler were deliberately built so that its current analysis state stayed addressable after an error, how would the resulting developer–compiler interaction proceed, and what would be required to implement it? The small language and pipeline exist specifically to let this question be answered by construction rather than by argument.

#### **4.6 Nature of the Innovation**

It is worth being precise about what is, and is not, being proposed as novel. The traditional interaction pattern is linear: an error occurs, the developer reads the message, edits the source, and recompiles, with each cycle starting from nothing. Interactive Compiler Interface inserts an additional stage into this cycle: after an error, the developer can explain, inspect, or request suggestions before choosing to apply a fix and recompile, all resolved against retained compiler state rather than against a freshly re-derived guess. The project does not claim that no existing compiler or development environment offers diagnostics, quick fixes, or code navigation, many do, and Interactive Compiler Interface does not attempt to replicate or replace that tooling. Its contribution is narrower and specific to compiler design: representing the compiler's active analysis context as an addressable, queryable object for the duration of a session, and building the small pipeline and session layer needed to make that representation real rather than only proposed. The innovation, in other words, sits in the representation and interaction model built around the pipeline, not in a new parsing, type-checking, or code-generation algorithm.

### **5. Objectives**

The objectives describe what the overall Interactive Compiler Interface project intends to achieve across its full planned scope; they are not a status checklist for what has already been built, and completion against each objective is assessed against the full planned system rather than the Review 1 prototype alone. The objectives, carried forward from Phase 1 with only grammatical refinement, are:

1. Design a small programming language with a precise lexical and syntactic specification, sufficient to exercise the full compiler pipeline.
2. Develop a compiler-style pipeline covering lexical analysis, syntax analysis, AST construction, semantic analysis, symbol-table management, intermediate-representation generation, and execution.
3. Represent compiler diagnostics as structured data , including phase, diagnostic kind, source location, and relevant compiler-state context , rather than as plain diagnostic strings.
4. Provide an interface through which a developer can query the currently active compiler state following a diagnostic.
5. Maintain the current diagnostic and compilation context within a short session, so that follow-up commands are resolved against the same analysis state that produced the diagnostic.
6. Offer deterministic explanations and situation-specific alternatives derived directly from the compiler's structured information, rather than from free-form generated text.
7. Implement a small set of explicitly supported, safe source edits, together with automatic recompilation and verification once a repair is applied.
8. Demonstrate a genuine developer–compiler interaction mode while keeping the project's scope bounded, so that it does not grow into a general-purpose IDE, language server, or AI coding assistant.

## **6. Scope**

The full Interactive Compiler Interface design and the Review 1 prototype cover different amounts of ground. This section first states the overall project scope , the complete system the project is working toward , and then states, separately, the deliberately narrower scope of the Review 1 prototype that has been frozen for this milestone.

### **6.1 Overall Project Scope**

The complete, planned Interactive Compiler Interface system is intended to support the following. This is the target for the project as a whole, across all implementation phases, not a description of what Review 1 alone delivers.

#### **6.1.1 Functional Scope**

- A small imperative language with variable declarations, assignments, expressions, output statements, conditional statements, and iterative (loop) constructs.
- Lexical, syntactic, and semantic processing of programs written in that language.
- Construction of an abstract syntax tree and an associated symbol table.
- Generation and inspection of a simple intermediate representation, and execution of the resulting program.
- Structured, interactive reporting through a small set of session commands.
- Preservation of diagnostic and compiler-state context across a sequence of related commands within one session.

- A bounded set of candidate explanations and repair operations associated with a given diagnostic.
- Automatic recompilation once a repair has been applied to the source.

### **6.1.2 Language Scope**

The language deliberately has far fewer features than a general-purpose programming language. It supports the primitive types `int`, `float`, `bool`, and `string`; arithmetic, relational, and logical operators; variable declaration and assignment; `print` statements; `if/else` conditional blocks; and `while` loops. The complete lexical and grammatical specification is given in Section 9.3, Language Specification.

### **6.1.3 Interaction Scope**

The interaction vocabulary is intentionally small and fixed. The primary commands allow the developer to request an explanation, ask why a rule was violated, inspect a symbol, request candidate fixes, and apply a selected fix. These commands operate strictly on the current compiler state held by the session manager; they are not a general natural-language interface, and no command is accepted outside this fixed, explicitly supported set.

## **6.2 Review 1 Prototype Scope**

The Review 1 prototype does not attempt the overall project scope described in Section 6.1. It is deliberately restricted to the smallest language subset and the smallest set of pipeline and interaction components needed to demonstrate one complete instance of the Interactive Compiler Interface interaction cycle. The restriction is intentional rather than a sign of incompleteness: it keeps the first milestone controlled and verifiable, while the broader features remain explicitly planned for subsequent implementation phases.

For Review 1, the prototype is limited to:

- A single language construct: an `int` variable declaration with an initializing expression and a statement terminator, for example `int x = 10;`
- A single diagnostic kind: a missing statement terminator (`MISSING_TOKEN`), detected during syntax analysis.
- A single deterministic repair: insertion of the missing `';`' at the recorded source location.
- Exactly three interaction commands: `explain`, `suggest fixes`, and `apply 1`.
- One retained diagnostic per session, rather than a general multi-diagnostic session model.
- Genuine recompilation of the corrected source through the same lexer/parser path, without semantic analysis, IR generation, or execution.

Features belonging to the overall project scope but not required for Review 1, additional statement types, additional data types, semantic analysis, the symbol table, IR generation, execution, additional diagnostic kinds and repairs, and the `why` and `show symbols` commands, remain part of the planned full system and are addressed in later implementation phases, as detailed in Section 12.4.

## 6.3 Boundaries and Constraints

To keep the project realistic for an individual semester deliverable, the following are explicitly out of scope for the project as a whole, including all later phases:

- Interactive Compiler Interface is a controlled, sandbox-scale compiler, not a production compiler or a replacement for an existing compiler toolchain.
- It is not a complete Integrated Development Environment (IDE) or a language server.
- Unrestricted, large-scale automatic modification of source code is not supported.
- Large-language-model-based generation and unrestricted autonomous code repair are not part of the design; only explicitly defined, deterministic repair rules are used.
- The initial implementation targets a single controlled language with a command-line interface; graphical tooling is not part of Phase 1.

## 6.4 Technical Feasibility

Each pipeline stage of the overall project, lexer, parser, AST, semantic analyzer, IR generator, and interpreter, is a well-understood component with established implementation techniques, and the interaction layer adds a bounded command set on top of state these components already compute. Because the target language is deliberately small, the full pipeline is achievable within one individual semester project. As a first step toward that goal, the Review 1 prototype described in Section 12 is defined to establish technical feasibility concretely, through one working vertical slice, lexer, parser, structured diagnostic, session state, interaction, repair, and recompilation, for a single, well-defined scenario.

# 7. Background Study

## 7.1 Compiler Architecture

Compiler construction conventionally decomposes source processing into a sequence of phases, each responsible for a specific transformation or check. Lexical analysis converts a character stream into classified tokens; syntax analysis checks the token stream against a grammar and builds a structural representation of the program; semantic analysis checks meaning-level rules such as type compatibility and declaration validity; and later phases may build an intermediate representation and either interpret it directly or generate target code. This phase separation is well established and is the structure Interactive Compiler Interface's own pipeline follows.

## 7.2 Compiler Diagnostics

A diagnostic is the mechanism by which a compiler reports a problem found during analysis. A useful diagnostic typically identifies the source location of the problem, the nature of the problem, and, where relevant, what was expected versus what was found. In most compilers this information is formatted into a text message at the point of failure and is not afterwards exposed, through the diagnostic interface, in a form the developer can query further. Interactive Compiler Interface's proposed departure is to keep this



same information as a structured data object for the duration of a session, so it can support further queries rather than being reduced immediately to a printed string.

### **7.3 Existing Developer–Compiler Interaction**

Modern development environments already provide diagnostics inline, allow navigation to the reported location, and in many cases offer automated quick fixes for common problems. This project does not claim that such tooling is absent or inadequate; it assumes the opposite, and does not attempt to duplicate features such as project-wide navigation, refactoring, or IDE-level tooling. The distinction Interactive Compiler Interface explores is narrower: whether the compiler's active analysis context, not just a fix suggestion, but the diagnostic state itself, can be made into an object the developer interacts with directly and repeatedly within a single compilation session.

### **7.4 Conceptual Gap Explored**

The gap under examination is representational and interactional rather than algorithmic. Interactive Compiler Interface's lexer, parser, and (in later phases) semantic analyzer perform the same categories of analysis as any small compiler; nothing about the analysis techniques themselves is claimed to be new. What the project investigates is what happens to the results of that analysis after an error is found, specifically, whether treating them as addressable session state, rather than as an intermediate value the diagnostic interface does not expose, changes how a developer can work through a compiler error.

### **7.5 Relevant Technologies and Concepts**

Several existing ideas are relevant context for this project, even though none of them is being reproduced directly. Read–eval–print loops (REPLs) keep interpreter state alive across successive inputs, which is conceptually related to Interactive Compiler Interface's session model, though a REPL retains program state rather than compiler diagnostic state. Language Server Protocol implementations expose structured diagnostic and symbol information to editors over a defined protocol, which is close in spirit to representing diagnostics as structured data, though LSP is oriented toward editor tooling rather than a conversational session with the compiler itself. IDE quick-fix systems apply a small number of predefined code transformations in response to a specific diagnostic, which is structurally similar to Interactive Compiler Interface's bounded repair actions. Interactive Compiler Interface combines elements of these ideas, retained session state, structured diagnostics, and bounded automatic repair, around a compiler pipeline built specifically to make the resulting interaction model observable end to end.

### **7.6 Sources**

- Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D., *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson.
- Appel, A. W., *Modern Compiler Implementation in Java*, 2nd ed., Cambridge University Press.
- Grune, D., van Reeuwijk, K., Bal, H. E., Jacobs, C. J. H., and Langendoen, K., *Modern Compiler Design*, 2nd ed., Springer.
- *Compiler Design Laboratory Project Instruction Manual*, course-provided Phase 1 project guidelines.

## 8. Compiler Design Concepts Involved

Interactive Compiler Interface is grounded in the standard set of Compiler Design concepts covered in this course; no additional theory outside the syllabus is required. Each pipeline component has a direct, specific role both in performing conventional compiler analysis and in supplying the structured information that the interaction layer subsequently exposes. The table below states, for each concept as it applies to the overall planned system, what it does in a conventional compiler, its specific role in Interactive Compiler Interface, and how it contributes to the interaction model under study.

| Compiler Concept                        | What It Does in the Compiler                                                                                                           | Role in Interactive Compiler Interface                                                               | Contribution to the Interaction Model                                                                      |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Lexical Analysis</b>                 | Converts source characters into classified tokens (keywords, identifiers, literals, operators, delimiters) and reports lexical errors. | Produces the token stream consumed by the parser and detects malformed tokens.                       | Supplies precise source locations and token categories used in lexical diagnostics.                        |
| <b>Syntax Analysis</b>                  | Validates the token stream against the language grammar and builds a parse structure.                                                  | Detects structural errors such as a missing statement terminator or malformed expression.            | Generates the syntax diagnostics that most interaction-model commands are demonstrated against.            |
| <b>Abstract Syntax Tree (AST)</b>       | Represents the program's structure independently of its surface syntax.                                                                | Serves as the structural representation passed to semantic analysis and IR generation.               | Provides the structural context referenced by explain and inspection commands.                             |
| <b>Semantic Analysis</b>                | Checks declarations, scoping rules, and type compatibility of expressions and statements.                                              | Confirms that a syntactically valid program is also meaningful before execution.                     | Produces semantic diagnostics and the reasoning shown by why and explain.                                  |
| <b>Symbol Table</b>                     | Stores identifiers together with their types and scope information.                                                                    | Tracks declared variables across the small language's single global and block scopes.                | Backs the show symbols command and supplies context for explanations that reference a specific identifier. |
| <b>Intermediate Representation (IR)</b> | Represents valid statements and expressions in a simple, lower-level form suitable for execution.                                      | A compact three-address-style form generated after successful semantic analysis.                     | Offers an inspectable, structured view of a compiled program independent of the interpreter.               |
| <b>Execution/ Interpretation</b>        | Evaluates a valid program and produces its runtime result.                                                                             | A tree-walking interpreter executes the IR (or AST) of a program that has passed all earlier phases. | Confirms successful recompilation and produces the final result shown to the developer.                    |
| <b>Error Detection</b>                  | Identifies and classifies failures at each pipeline phase.                                                                             | Converts a raw failure into a typed, structured diagnostic object rather than a formatted string.    | Is the origin point of the diagnostic object that the session manager subsequently retains.                |

|                                |                                                                                     |                                                                                                                |                                                                                                      |
|--------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Error Recovery / Repair</b> | Defines how a compiler may respond to, or partially recover from, a detected error. | Supplies a small, explicitly enumerated set of deterministic repair actions tied to specific diagnostic kinds. | Backs the suggest fixes and apply commands, and drives the recompilation step that follows a repair. |
|--------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|

For the Review 1 prototype specifically, only Lexical Analysis, Syntax Analysis, Error Detection, and Error Recovery / Repair are within scope, against the single missing-semicolon scenario described in Section 12. Abstract Syntax Tree construction is used only to the extent the parser requires it to detect the error; Semantic Analysis, the Symbol Table, Intermediate Representation, and Execution / Interpretation remain part of the planned full system and are not required for, or claimed by, the Review 1 demonstration.

## 9. Proposed Methodology

### 9.1 Full System Methodology

The system is organised as two connected modules: a compiler pipeline module and an interaction layer module. The compiler module processes source code in the conventional way, but additionally exposes its internal structures (tokens, AST, symbol table, diagnostics) to the layer above rather than keeping them private to the compilation run. The interaction layer module receives developer commands and resolves them against the state of the current session. This describes the end-to-end processing method for the complete, planned Interactive Compiler Interface system:

1. Accept source code and begin a compilation session.
2. Perform lexical analysis and produce a token stream.
3. Perform syntax analysis and construct the AST.
4. Perform semantic analysis using the symbol table.
5. Generate a simple intermediate representation for inspection.
6. Execute the program if it is valid.
7. If an error occurs at any stage, construct a structured diagnostic describing it.
8. Store the diagnostic, together with the relevant compiler state, in the session.
9. Accept developer commands and resolve each one against the current session state.
10. Return an explanation, an inspection result, a decision, or a repair result as appropriate.
11. If a developer applies a repair, update the source accordingly and re-run the pipeline from step 2.

### 9.2 Review 1 Vertical Slice

The Review 1 prototype is defined as one path through the methodology above, restricted to the components and steps required for the missing-semicolon scenario. This vertical slice was selected because it exercises the central interaction model end to end , compiler analysis, structured diagnostic

creation, session retention, developer query, deterministic repair, source transformation, and recompilation , while keeping implementation scope manageable for an individual project within the Review 1 timeframe. The steps in scope for Review 1 are:

Source

|

v

Lexer

|

v

Parser

|

v

Missing-semicolon detection

|

v

Structured Diagnostic

|

v

Session State

|

v

Interaction command (explain / suggest fixes / apply 1)

|

v

Explanation / Suggested Repair

|

v

Repair Engine

|

v

Source Modification

|

v

Recompilation

|

v

Success

Semantic analysis, symbol-table population, IR generation, and execution are part of the full-system methodology above but are not exercised by this vertical slice, since the missing-semicolon scenario is resolved entirely within lexical and syntax analysis.

### 9.3 Requirement-to-Module Mapping

| Design Need                              | Relevant Compiler Module | Review 1 Status                                        |
|------------------------------------------|--------------------------|--------------------------------------------------------|
| <b>Recognize source elements</b>         | Lexer                    | Included in Review 1                                   |
| <b>Validate language structure</b>       | Parser                   | Included in Review 1                                   |
| <b>Represent program structure</b>       | AST                      | Minimal use only, as needed by the parser              |
| <b>Check meaning and types</b>           | Semantic Analyzer        | Planned for later phases                               |
| <b>Track identifiers and types</b>       | Symbol Table             | Planned for later phases                               |
| <b>Expose intermediate processing</b>    | IR Generator             | Planned for later phases                               |
| <b>Execute valid programs</b>            | Interpreter              | Planned for later phases                               |
| <b>Represent compiler problems</b>       | Diagnostic Manager       | Included in Review 1                                   |
| <b>Interpret developer commands</b>      | Interaction Manager      | Included in Review 1 (explain, suggest fixes, apply 1) |
| <b>Preserve current compiler context</b> | Session Manager          | Included in Review 1                                   |
| <b>Provide bounded corrections</b>       | Repair Engine            | Included in Review 1 (single repair action)            |

### 9.4 Language Specification

The language specification below is carried forward from the Phase 1 submission without technical change, since no inconsistency with it was identified. It describes the full, planned language. Section 9.4.7 separately identifies the subset of this language actually used by the Review 1 prototype.

#### 9.4.1 Data Types

| Type          | Purpose               | Example |
|---------------|-----------------------|---------|
| <b>int</b>    | Integer values        | 10      |
| <b>float</b>  | Floating-point values | 3.14    |
| <b>bool</b>   | Boolean values        | true    |
| <b>string</b> | Text values           | "hello" |

#### 9.4.2 Keywords

int, float, bool, string, print, if, else, while, true, false

#### 9.4.3 Operators and Delimiters

| Category          | Symbols         |
|-------------------|-----------------|
| <b>Arithmetic</b> | + - * / %       |
| <b>Relational</b> | < <= > >= == != |

|                   |             |
|-------------------|-------------|
| <b>Logical</b>    | &&    !     |
| <b>Assignment</b> | =           |
| <b>Delimiters</b> | ( ) { } ; , |

#### 9.4.4 Identifiers and Literals

- Identifiers begin with a letter or an underscore and may contain letters, digits, and underscores thereafter.
- Integer literals consist of one or more decimal digits.
- Floating-point literals contain a decimal point together with digits on at least one side.
- String literals are enclosed in double quotation marks.
- Boolean literals are the reserved words true and false.

#### 9.4.5 Token Categories

| Token Category        | Representative Tokens                            |
|-----------------------|--------------------------------------------------|
| <b>KEYWORD</b>        | int, float, bool, string, print, if, else, while |
| <b>IDENTIFIER</b>     | x, total, message                                |
| <b>INT_LITERAL</b>    | 10, 42                                           |
| <b>FLOAT_LITERAL</b>  | 3.14, 0.5                                        |
| <b>STRING_LITERAL</b> | "hello"                                          |
| <b>BOOL_LITERAL</b>   | true, false                                      |
| <b>OPERATOR</b>       | +, -, *, /, =, ==, <, &&                         |
| <b>DELIMITER</b>      | (, ), {, }, ;                                    |

#### 9.4.6 Grammar (EBNF)

```

program ::= { statement } ;
statement ::= declaration
           | assignment
           | print_stmt
           | if_stmt
           | while_stmt ;
declaration ::= type IDENTIFIER "=" expression ";" ;
assignment ::= IDENTIFIER "=" expression ";" ;
print_stmt  ::= "print" "(" expression ")" ";" ;
if_stmt     ::= "if" "(" expression ")" block
              [ "else" block ] ;
while_stmt  ::= "while" "(" expression ")" block ;
block       ::= "{" { statement } "}" ;
type        ::= "int" | "float" | "bool" | "string" ;
expression  ::= logical_or ;
logical_or  ::= logical_and { "|" logical_and } ;

```

```

logical_and ::= equality { "&&" equality } ;
equality   ::= relational { ("==" | "!=") relational } ;
relational ::= additive { ("<" | "<=" | ">" | ">=") additive } ;
additive   ::= multiplicative { ("+" | "-") multiplicative } ;
multiplicative ::= unary { ("*" | "/" | "%") unary } ;
unary      ::= ("!" | "-") unary | primary ;
primary    ::= IDENTIFIER
              | literal
              | "(" expression ")" ;
literal    ::= INT_LITERAL
              | FLOAT_LITERAL
              | STRING_LITERAL
              | BOOL_LITERAL ;

```

#### 9.4.7 Review 1 Prototype Language Subset

The Review 1 prototype does not implement the full grammar above. It supports only the subset of the language needed for the missing-semicolon scenario:

- An int type declaration.
- A single identifier.
- The assignment operator '='.
- An integer literal.
- The statement terminator ';'.

Constructs such as if, else, while, float, bool, string, and compound expressions are part of the grammar above but are not required, and are not claimed, to be supported by the Review 1 prototype. They remain scope for later implementation phases, consistent with Section 6.2.

### 9.5 Diagnostic and Session Model

A compiler problem is represented as a diagnostic object suitable for further processing, rather than as a string intended only for display. The structure below shows the fields carried by a diagnostic in the full, planned model:

```

Diagnostic {
  phase // e.g. LEXICAL, SYNTAX, SEMANTIC
  type  // diagnostic kind, e.g. MISSING_TOKEN
  line, column // source location
  symbol // identifier involved, if any
  expected // token/type expected by the rule
  actual   // token/type actually found

```

```
reason // short structured explanation
actions[] // candidate repair actions, if any
}
```

For the Review 1 prototype, the diagnostic object above is populated concretely for the single missing-semicolon case.

```
Diagnostic {
  phase: SYNTAX
  type: MISSING_TOKEN
  line: 1
  column: 11
  symbol: "x"
  expected: ";"
  actual: <end of line 1>
  reason: "Declaration of 'x' requires ';'
  actions: [ INSERT_SEMICOLON ]
}
```

The session manager retains the active source, the active diagnostic, the portion of compiler state relevant to that diagnostic, and any candidate repair actions, for as long as the developer continues to interact with the same error. For the Review 1 prototype this means, specifically, that the single Diagnostic object created by the parser is the same object that explain, suggest fixes, and apply 1 all read from and act upon, the three commands are not independent outputs, but three views onto one retained piece of session state. This is a central proof point of Interactive Compiler Interface and is examined further in Section 12.

## 9.6 Interaction Model and Repair Methodology

The full interaction vocabulary is deliberately small, and which commands are available at a given moment depends on the current diagnostic or compiler state, for example, suggest fixes is only meaningful once a diagnostic with candidate actions exists. Explanations are deterministic: the same diagnostic always produces the same explanation, generated from the structured diagnostic fields rather than from freely generated text. Repairs are expressed as explicit actions, each carrying a short description, a precise source location, and the replacement text to insert; no transformation outside this predefined set is permitted.

The full vocabulary includes explain, why, show symbols, suggest fixes, and apply. Of these, why and show symbols depend on semantic analysis and symbol-table population and are not part of the Review 1 prototype; they remain planned for later phases, as noted in Section 12.4. The Review 1 prototype is defined to support exactly three commands: explain, suggest fixes, and apply 1. The representative exchange below is the canonical Review 1 demonstration, reproduced formally as an execution trace in Section 12.5:



COMPILER: Error at line 1: expected ';'

USER: explain

COMPILER: The declaration of 'x' is missing the statement

COMPILER: terminator ';' required by the language grammar.

USER: suggest fixes

COMPILER: [1] Insert ';' at the end of line 1.

USER: apply 1

COMPILER: Fix applied. Recompiling...

COMPILER: Compilation successful.

## 9.7 Deterministic Repair Constraints

Every repair action is tied to a specific diagnostic kind and is limited to a small, local source change (for example, inserting a missing delimiter at a known location). No repair action modifies source code outside the span associated with its diagnostic, and no repair is generated dynamically at runtime, each is drawn from a predefined table of actions associated with diagnostic kinds. This keeps repair behaviour predictable and directly traceable to a specific rule in the language grammar or semantic checks.

For the Review 1 prototype, the single repair action is defined explicitly as follows, and no other repair behaviour is claimed:

Diagnostic: MISSING\_TOKEN

Expected: ";"

Repair: INSERT ";"

Location: End of the affected declaration

(the source location recorded on the diagnostic)

Operation: Insert the missing semicolon at that location.

This repair is deterministic, predefined, local to the single affected line, tied to exactly one diagnostic kind, and only ever performed when explicitly selected by the developer through apply 1.

## 9.8 Recompilation Process and Context Preservation

Once a repair action is applied, the session manager updates the stored source text at the location specified by the action and invokes the compiler pipeline again from lexical analysis. If the recompiled program succeeds, the session reports the successful result; if a new diagnostic is produced, it replaces the previous one as the active context, and the interaction cycle continues from that new state. This ensures that context is preserved across the full explain → suggest → apply → recompile cycle rather than only within a single command exchange.

Applying the repair is not, by itself, treated as sufficient evidence that the repair succeeded. Modifying the source text only shows that a string replacement occurred; it does not by itself show that the resulting program is syntactically valid. For this reason, the recompilation step is a required, not optional, part of the interaction cycle: after a repair is applied, the corrected source is genuinely resubmitted to the lexer and parser, and success is reported only if that resubmitted compilation actually completes without error. For the Review 1 scenario this means:

```
int x = 10      -> diagnostic (missing ';')
apply 1
  -> int x = 10;  -> lexer -> parser -> successful compilation
```

The report does not claim that the corrected program is executed, since execution is outside the Review 1 prototype scope defined in Section 6.2; successful recompilation here refers specifically to successful lexical and syntax analysis of the corrected source.

## 10. System Architecture

### 10.1 High-Level Architecture (Planned Full System)

The architecture places the interaction layer above a controlled compiler pipeline. The developer communicates only with the interaction layer; the interaction layer, in turn, reads structured state produced by the compiler pipeline through the session manager, rather than by reaching into the pipeline directly. This diagram describes the complete, planned architecture; Section 10.6 shows separately the narrower path in scope for Review 1.

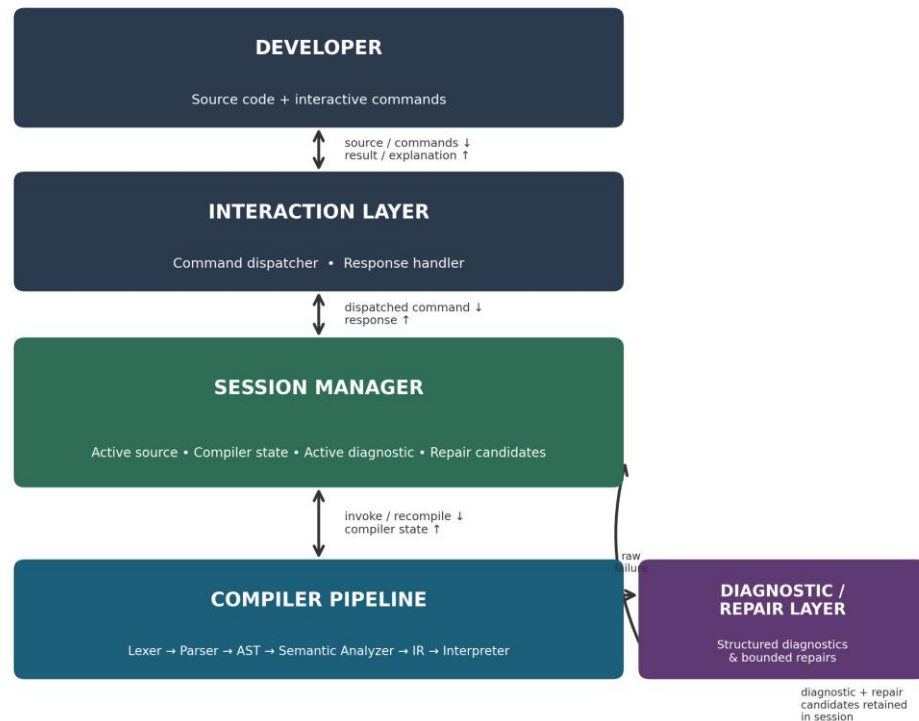
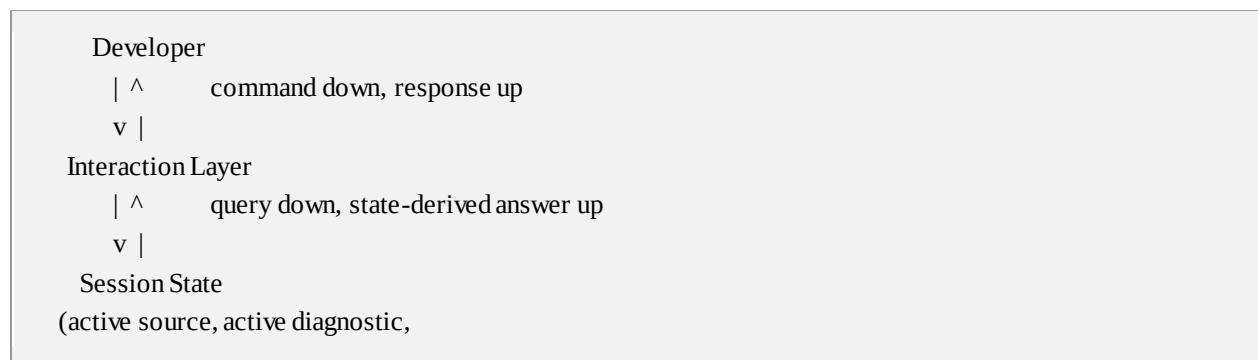


Figure 1. High-level architecture of the planned full Interactive Compiler Interface system. The two-way relationship holds between the developer, the interaction layer, and the retained session state: a request travels downward and a state-bearing response travels back up. The compiler pipeline below the session layer is invoked in the conventional forward direction; what returns from it is diagnostic and context information, not a reversal of the pipeline.

**What Makes the Interaction Two-Way.** A conventional compiler interaction is one-way: the developer submits source, the compiler analyses it, and a diagnostic is produced and printed; the exchange ends there, and no further request against that analysis is possible. Interactive Compiler Interface’s interaction is two-way in a specific, checkable sense: after the diagnostic is produced, the developer can send a further request, explain, suggest fixes, or apply 1, back against the compiler’s retained analysis state, and the compiler answers that request using the same retained state rather than by re-deriving a fresh guess. Concretely, the reverse direction consists of the developer’s query travelling from the Interaction Layer down to the Session Manager, the Session Manager resolving it against the active diagnostic (invoking the Compiler Pipeline again only when a repair is applied), and a state-derived response travelling back up through the Interaction Layer to the developer. This is what a viva examiner should be shown when asked what exactly makes the model two-way, rather than the title alone.



```

retained compiler context)
  | ^      source down, diagnostic + context up
  v |
Compiler Pipeline
Lexer -> Parser -> AST -> Semantic -> IR -> Interpreter
(invoked in the forward direction only)

```

Stated operationally, and independently of the diagram, one full cycle of the interaction proceeds as follows:

1. The developer submits source code.
2. The compiler pipeline analyses it in the conventional forward direction.
3. A pipeline stage produces a structured diagnostic describing the failure.
4. The session manager retains that diagnostic together with the relevant compiler context.
5. The developer issues a subsequent command about that same diagnostic.
6. The interaction layer resolves the command against the retained state rather than re-deriving an answer.
7. A state-derived response travels back up to the developer.
8. If, and only if, a repair is explicitly selected, the source is modified and the compiler pipeline is invoked again from lexical analysis.

The compiler pipeline itself is therefore not bidirectional. Each phase still runs in the usual order, and nothing flows backwards through the phases. What travels upward is the diagnostic and context information those phases produce, and what makes the model two-way is the loop formed between the developer, the interaction layer, and the retained session state that sits above the pipeline.

| Layer                           | Main Responsibility                                                      |
|---------------------------------|--------------------------------------------------------------------------|
| <b>Developer</b>                | Source code and interactive commands.                                    |
| <b>Interaction Layer</b>        | Command dispatcher and response handler.                                 |
| <b>Session Manager</b>          | Active source, compiler state, active diagnostic, and repair candidates. |
| <b>Compiler Pipeline</b>        | Lexer → Parser → AST → Semantic Analyzer → IR → Interpreter.             |
| <b>Diagnostic/ Repair Layer</b> | Structured diagnostics and bounded repair operations.                    |
| <b>Result</b>                   | Explanation, alternatives, applied repair, and recompilation result.     |

## 10.2 Compiler Pipeline

Within the Compiler Pipeline layer, source code passes through the conventional sequence of phases shown below. This sequence is unchanged by the presence of the interaction layer; it is the same pipeline a conventional compiler for this language would use. The Review 1 prototype covers only the first two stages of this sequence, Lexer and Parser, sufficient to detect the missing-semicolon diagnostic; AST is

used only minimally as required by the parser, and Semantic Analyzer, IR, and Interpreter remain planned.

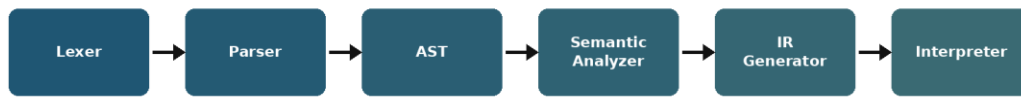


Figure 2. Compiler pipeline stages of the planned full system.

### 10.3 Module Responsibilities

| Module                     | Responsibility                                                     | Review 1 Status                                     |
|----------------------------|--------------------------------------------------------------------|-----------------------------------------------------|
| <b>Lexer</b>               | Converts source characters into classified tokens.                 | In Review 1 scope (language subset)                 |
| <b>Parser</b>              | Consumes tokens according to the grammar and constructs the AST.   | In Review 1 scope (missing-semicolon detection)     |
| <b>AST</b>                 | Stores the hierarchical structural representation of the program.  | Minimal use only, as required by the parser         |
| <b>Semantic Analyzer</b>   | Checks declarations, types, and semantic rules.                    | Planned for later phases                            |
| <b>Symbol Table</b>        | Tracks identifiers, types, and scope-related information.          | Planned for later phases                            |
| <b>IR Generator</b>        | Produces a simple intermediate representation for inspection.      | Planned for later phases                            |
| <b>Interpreter</b>         | Executes valid programs and produces the runtime result.           | Planned for later phases                            |
| <b>Diagnostic Manager</b>  | Converts a detected failure into a structured compiler diagnostic. | In Review 1 scope (MISSING_TOKEN case)              |
| <b>Interaction Manager</b> | Maps developer commands to compiler-state queries or actions.      | In Review 1 scope (explain, suggest fixes, apply 1) |
| <b>Session Manager</b>     | Preserves the active compilation context across commands.          | In Review 1 scope (single retained diagnostic)      |
| <b>Repair Engine</b>       | Generates and applies supported deterministic repairs.             | In Review 1 scope (semicolon-insertion action)      |

The Review 1 Status column above states which modules are within the scope of the Review 1 vertical slice. It is a statement of scope, not of completion: the verified implementation status of each module is recorded separately in Section 12.4.

### 10.4 Error Interaction Workflow

1. The developer submits source code to begin a session.
2. A compiler pipeline stage detects an error.
3. The Diagnostic Manager converts the failure into a structured diagnostic object.
4. The Session Manager stores the active diagnostic and the relevant compiler-state context.

5. The developer issues a command about the active problem.
6. The Interaction Manager resolves the command against the stored session state.
7. The system returns an explanation, an inspection result, a list of alternatives, or a repair action, as appropriate to the command.
8. If a repair is selected, the Repair Engine updates the source and the pipeline is invoked again from lexical analysis.
9. The result of the new compilation becomes the current compiler state for any further commands.

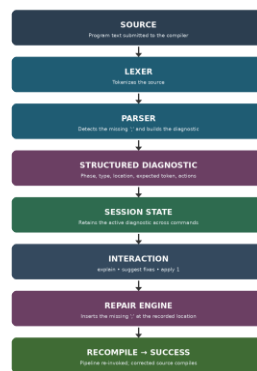
This workflow describes both the full planned system and the Review 1 prototype; the prototype follows exactly this sequence, but with the pipeline in step 2 limited to lexical and syntax analysis, and with a single diagnostic kind and a single repair action available in steps 3–8.

## 10.5 Relationship Between the Compiler Pipeline and the Interaction Layer

The architecture is designed so that the conventional compiler pipeline is not modified by the presence of the interaction layer. The Interaction Layer does not perform lexical analysis, parsing, semantic analysis, or code execution itself; it only reads and acts upon state that the pipeline has already produced, through the Session Manager. This separation allows the compiler pipeline and the interaction code to be understood, tested, and reasoned about independently, while still functioning together as a single system. Preserving this separation is central to the experiment: the interaction mechanism is added around existing compiler components rather than woven into them, so that the pipeline continues to behave as a conventional compiler pipeline regardless of which interaction commands are later layered on top of it.

## 10.6 Review 1 Prototype

The full architecture in Section 10.1 and the full pipeline in Section 10.2 describe the system Interactive Compiler Interface is being built toward. Review 1 covers only the vertical path required to demonstrate the central interaction concept: source, lexer, parser, structured diagnostic, session, interaction, repair, and recompilation. The remaining modules , semantic analyzer, symbol table, IR generator, and interpreter , form the planned complete system and are not part of the Review 1 implementation. This narrower path is shown separately below.



*Figure 3. Review 1 Prototype Vertical Slice , the subset of the full architecture in scope for the Review 1 demonstration.*

## 11. Technology Stack

| Component                          | Technology                              | Purpose                                                           |
|------------------------------------|-----------------------------------------|-------------------------------------------------------------------|
| <b>Programming Language</b>        | Python 3.x                              | Compiler pipeline, interaction layer, and session management.     |
| <b>Lexer / Parser</b>              | Hand-written (Python)                   | Direct control over tokens, grammar, and diagnostic generation.   |
| <b>AST</b>                         | Python classes / data structures        | Structured representation of parsed programs.                     |
| <b>Semantic Analysis</b>           | Python (planned)                        | Type checking and declaration/scope validation, for later phases. |
| <b>Intermediate Representation</b> | Custom three-address-style IR (planned) | Inspectable, lower-level compiler state, for later phases.        |
| <b>Execution</b>                   | Tree-walking interpreter (planned)      | Controlled execution of valid programs, for later phases.         |
| <b>Interface</b>                   | Command-line interface                  | Developer–compiler interaction for Phase 1.                       |
| <b>Version Control</b>             | Git                                     | Source-code versioning and development history.                   |

Rows marked “planned” reflect components of the overall technology stack that are not required by, and are not part of, the Review 1 prototype, which uses only the hand-written lexer, parser, diagnostic model, session manager, interaction manager, repair engine, and command-line interface described in Section 12.

### 11.1 Technology Selection Rationale

Python was chosen because it allows the compiler pipeline to be written compactly while keeping internal data representations (tokens, AST nodes, diagnostic objects) easy to inspect during development and demonstration, a useful property for a project whose central claim rests on making internal compiler state visible. A hand-written lexer and parser were chosen over a parser-generator tool because they give direct, fine-grained control over exactly what information is captured at the point of a syntax error, which is necessary for constructing detailed structured diagnostics rather than only accept/reject results. A command-line interface is sufficient for Phase 1 because the object of evaluation is the interaction model and the underlying compiler mechanism, not graphical presentation; a CLI keeps the developer-facing surface simple enough that the interaction commands themselves remain the focus of the demonstration.

### 11.2 Development Practices

The implementation is organised around modular programming practice, with the lexer, parser, semantic analyzer, IR generator, interpreter, diagnostic manager, session manager, and repair engine kept as separate, independently testable components, consistent with the module boundaries in Section 10.3. Variable and function names are chosen to be descriptive of their compiler-design role (for example, `tokenize`, `parse_statement`, `lookup_symbol`), and comments are used at points where the logic is non-obvious, such as the token-matching rules in the parser and the repair-selection logic in the repair engine. The project is version-controlled with Git, with commits corresponding to individual pipeline

components, and a simple project folder structure separates the compiler pipeline, the interaction layer, and test source programs.

## 12. Initial Prototype

### 12.1 Prototype Objective

The goal of the Review 1 prototype is to demonstrate the central Interactive Compiler Interface interaction idea, explain, suggest fixes, apply, recompile, through one complete vertical slice of the system, before the full language and pipeline described in Sections 9–11 are built out. Rather than implementing many partial features across the whole planned system, the prototype deliberately restricts the language surface to what is needed to reach a full error → interaction → repair → recompilation cycle for one concrete, well-understood error.

The missing-semicolon case was selected because it provides the smallest complete vertical slice through the proposed interaction architecture. Although the language construct is simple, the scenario exercises the full interaction loop: compiler analysis, structured diagnostic creation, session retention, developer query, deterministic repair, source transformation, and recompilation. The scope is deliberately narrow, not because the underlying architecture is limited, but because a single, fully working instance of the interaction cycle is stronger evidence of feasibility at this stage than several partially working ones.

### 12.2 Review 1's Minimum Useful Demonstration

The exact chosen scenario for review 1's demonstration is:

SOURCE:

```
int x = 10
```

COMPILER: Error at line 1: expected ';'.

YOU: explain

COMPILER: The declaration of 'x' is missing the

COMPILER: statement terminator ';' required by

COMPILER: the language grammar.

YOU: suggest fixes

COMPILER: [1] Insert ';' at the end of line 1.

YOU: apply 1

COMPILER: Fix applied. Recompiling...



COMPILER: Compilation successful.

This scenario demonstrates the full chain the architecture is intended to support: compiler analysis → structured diagnostic → retained context → developer interaction → explanation → repair suggestion → selected repair → source modification → recompilation.

To make this demonstration reconstructable from the report alone, without relying on the live presentation, the exact input, expected behaviour, and expected outcome are restated here explicitly:

| Item                                   | Value                                     |
|----------------------------------------|-------------------------------------------|
| <b>Input</b>                           | int x = 10                                |
| <b>Expected diagnostic</b>             | Syntax error, missing ';', line 1         |
| <b>Interaction commands (in order)</b> | explain, then suggest fixes, then apply 1 |
| <b>Repair applied</b>                  | Insert ';' at the end of line 1           |
| <b>Expected corrected source</b>       | int x = 10;                               |
| <b>Verification step</b>               | Recompile the corrected source            |
| <b>Expected final result</b>           | Compilation successful.                   |

### 12.3 Prototype Components

The table below lists the components exercised by the Review 1 demonstration, the specific role each plays in it, and the evidence in this report that supports it. Figure references are in the corresponding screenshots, listed in Section 12.6, are captured from the working implementation.

| Component                  | Review 1 Role                                          | Evidence            |
|----------------------------|--------------------------------------------------------|---------------------|
| <b>Lexer</b>               | Tokenizes int, x, =, 10                                | Figure A            |
| <b>Parser</b>              | Detects the missing ';'.                               | Figure A / Figure B |
| <b>Diagnostic Model</b>    | Stores the structured syntax diagnostic (Section 9.5)  | Figure A / Figure B |
| <b>Session Manager</b>     | Retains the active diagnostic across commands          | Figure A / Figure B |
| <b>Interaction Manager</b> | Handles explain, suggest fixes, and apply 1            | Figure A / Figure B |
| <b>Repair Engine</b>       | Inserts the missing ';'.                               | Figure A / Figure B |
| <b>Recompilation</b>       | Re-runs the compiler and verifies the corrected source | Figure B / Figure C |

### 12.4 Implementation Status

This subsection is the definitive record of what has actually been built and verified at the time of submission, as distinct from what the Review 1 vertical slice is defined to cover. Every status below is to be replaced with a factual entry once the corresponding component has been implemented and observed working; no component is described as implemented anywhere in this report unless its row here says so.

| Component                    | Review 1 Status          | Evidence                     |
|------------------------------|--------------------------|------------------------------|
| <b>Lexer</b>                 | IMPLEMENTED AND VERIFIED | Figure A1/lexer.py           |
| <b>Parser</b>                | IMPLEMENTED AND VERIFIED | Figure A1/parser.py          |
| <b>Structured Diagnostic</b> | IMPLEMENTED AND VERIFIED | Figure A1/diagnostics.py     |
| <b>Session Manager</b>       | IMPLEMENTED AND VERIFIED | Figure A2/session.py         |
| <b>Interaction Manager</b>   | IMPLEMENTED AND VERIFIED | Figure A2/interaction.py     |
| <b>Repair Engine</b>         | IMPLEMENTED AND VERIFIED | Figure A2/repair.py          |
| <b>Recompilation</b>         | IMPLEMENTED AND VERIFIED | Figure B/Figure C/session.py |

The scope of each component above is as follows:

- Lexer , tokenizes the Review 1 language subset (the keyword `int`, an identifier, the assignment operator, an integer literal, and the statement terminator).
- Parser , detects the missing-semicolon syntax error and records its source location.
- Structured Diagnostic , the Diagnostic object of Section 9.5, populated for the `MISSING_TOKEN` case.
- Session Manager , retains the active source and the active diagnostic between successive commands.
- Interaction Manager , dispatches `explain`, `suggest fixes`, and `apply 1` against the retained diagnostic.
- Repair Engine , inserts the missing `';`' at the location recorded on the diagnostic.
- Recompilation , resubmits the corrected source through the lexer and parser and reports the result of that compilation.

#### Plan for Later Phases

- Full semantic analysis, including type checking across all supported types and scope rules for conditional and loop blocks.
- The symbol table, and the `show symbols` command that depends on it.
- IR generation and inspection for arbitrary valid programs, not only the Review 1 scenario.
- Execution of a valid program via the tree-walking interpreter.
- Additional diagnostic kinds and their corresponding repair actions (for example, unbalanced braces, undeclared identifiers, and type mismatches).
- The `why` command, which depends on semantic analysis not yet exercised by the prototype.

- Broader test coverage across the full grammar described in Section 9.4.

No feature listed above as planned for later phases is described as implemented anywhere in this report.

## 12.5 Prototype Execution Trace

The canonical Review 1 interaction, restated here as a formal execution trace corresponding to the internal steps in Section 9.2:

| Step | Internal Action                                                           |
|------|---------------------------------------------------------------------------|
| 1    | Source <code>int x = 10</code> is submitted.                              |
| 2    | Lexer tokenizes the source.                                               |
| 3    | Parser processes the tokens.                                              |
| 4    | Parser detects the missing semicolon.                                     |
| 5    | Parser creates a structured diagnostic (Section 9.5).                     |
| 6    | The diagnostic is retained in session state.                              |
| 7    | <code>explain</code> queries the retained diagnostic.                     |
| 8    | <code>suggest fixes</code> queries the available repair action.           |
| 9    | <code>apply 1</code> invokes the repair engine.                           |
| 10   | The repair engine modifies the source by inserting the missing semicolon. |
| 11   | The compiler pipeline is genuinely invoked again.                         |
| 12   | The corrected source is parsed successfully.                              |
| 13   | The system reports successful recompilation.                              |

This trace is not a scripted or mocked exchange in which `apply 1` is wired directly to a fixed “Compilation successful.” message. Steps 11–13 specifically require the compiler pipeline to be re-invoked on the corrected source and to genuinely complete lexical and syntax analysis before success is reported.

## 12.6 Evidence of Implementation

The report is intended to remain a complete, self-contained record of the Review 1 prototype after the live demonstration has concluded and after the code has been submitted separately. Accordingly, this subsection specifies exactly what each figure must show and why. Three figures are used, chosen so that each one carries a distinct piece of evidence rather than repeating the others. The figures themselves are to be captured from the working implementation and inserted in place of the placeholders below.

Figure A:

```
lexer.py
D: > Desktop > Projects > CompilerMate > compilermate > lexer.py > ...
1 from .tokens import Token, TokenType
2 class Lexer:
3     """Simple hand-written lexer for the minimal language.
4     Recognises the tokens needed for a single declaration.
5     """
6     KEYWORDS = {"int": TokenType.INT}
7     def __init__(self, source: str):
8         self.source = source
9         self.current = 0
10        self.line = 1
11        self.column = 1
12        self.tokens = []
13    def lex(self):
14        while not self._is_at_end():
15            self._skip_whitespace()
16            if self._is_at_end():
17                break
18            start_col = self.column
19            ch = self._peek()
20            if ch.isalpha():
21                lexeme = self._identifier()
22                token_type = self.KEYWORDS.get(lexeme, TokenType.IDENTIFIER)
23                self._add_token(token_type, lexeme, self.line, start_col)
24            elif ch.isdigit():
25                lexeme = self._number()
26                self._add_token(TokenType.INT_LITERAL, lexeme, self.line, start_col)
27            elif ch == '=':
28                self._advance()
29                self._add_token(TokenType.ASSIGN, '=', self.line, start_col)
30            elif ch == ';':
31                self._advance()
32                self._add_token(TokenType.SEMICOLON, ';', self.line, start_col)
33            else:
34                # Unknown character - skip it (the parser will later error)
35                self._advance()
36        # EOF token
37        self._add_token(TokenType.EOF, '', self.line, self.column)
38        return self.tokens
39    # ----- helpers -----
40    def _is_at_end(self):
41        return self.current >= len(self.source)
42    def _peek(self):
43        return self.source[self.current]
44    def _advance(self):
45        ch = self.source[self.current]
46        self.current += 1
47        self.column += 1
48        return ch
49    def _add_token(self, type_, lexeme, line, column):
50        self.tokens.append(Token(type_, lexeme, line, column))
51    def _skip_whitespace(self):
52        while not self._is_at_end() and self._peek().isspace():
53            self._advance()
54    def _identifier(self):
55        start = self.current
56        while not self._is_at_end() and (self._peek().isalnum() or self._peek() == '_'):
57            self._advance()
58        return self.source[start:self.current]
59    def _number(self):
60        start = self.current
61        while not self._is_at_end() and self._peek().isdigit():
62            self._advance()
63        return self.source[start:self.current]
```

Figure A1. Core lexer, parser, and structured diagnostic implementation.

```

lexer.py session.py X
D: > Desktop > Projects > CompilerMate > compilermate > session.py > ...
1 from dataclasses import dataclass, field
2 from typing import Optional, List
3 from .lexer import Lexer
4 from .parser import Parser
5 from .diagnostics import Diagnostic
6 from .repair import Repair
7 @dataclass
8 class Session:
9     """Keeps the compiler state across the interaction.
10     Holds the source code, lexer tokens, any diagnostic, the AST (if parsing succeeds),
11     and a list of repair candidates derived from the diagnostic.
12     """
13     source: str = ""
14     tokens: List = field(default_factory=list)
15     diagnostic: Optional[Diagnostic] = None
16     ast: Optional[object] = None
17     repair_candidates: List[Repair] = field(default_factory=list)
18     def compile(self) -> bool:
19         """Run the lexer and parser on the current ``source``.
20         Returns ``True`` when compilation succeeds (no diagnostic).
21         Populates ``tokens``, ``ast`` and ``diagnostic`` and generates repair candidates
22         for the specific MISSING_TOKEN case.
23         """
24         lexer = Lexer(self.source)
25         self.tokens = lexer.lex()
26         parser = Parser(self.tokens)
27         self.ast = parser.parse()
28         self.diagnostic = parser.diagnostic
29         # Reset repair list each compile
30         self.repair_candidates.clear()
31         if self.diagnostic:
32             if (
33                 self.diagnostic.type == "MISSING_TOKEN"
34                 and self.diagnostic.expected == ";"
35             ):
36                 repair = Repair(
37                     description="Insert missing semicolon",
38                     operation="INSERT",
39                     text=";",
40                     line=self.diagnostic.line,
41                     column=self.diagnostic.column,
42                 )
43                 self.repair_candidates.append(repair)
44         return self.diagnostic is None
45     def _offset_from_line_col(self, line: int, column: int) -> int:
46         """Convert a (line, column) pair to a zero-based string offset.
47         The MVP only supports a single-line source; any line > 1 is treated as the end.
48         """
49         if line != 1:
50             return len(self.source)
51         # column is 1-based; clamp to source length
52         offset = column - 1
53         return min(offset, len(self.source))
54     def apply_repair(self, index: int) -> bool:
55         """Apply the repair at ``index`` (0-based) to the current source.
56         Uses the helper to compute the insertion point, updates ``source`` and
57         re-runs ``compile``. Returns ``True`` if recompilation succeeds.
58         """
59         if index < 0 or index >= len(self.repair_candidates):
60             return False
61         repair = self.repair_candidates[index]
62         offset = self._offset_from_line_col(repair.line, repair.column)
63         self.source = self.source[:offset] + repair.text + self.source[offset:]
64         return self.compile()
65

```

Figure A2. Session state, interaction handling, and deterministic repair implementation.

Figure B:

```
Prashidika Upadhyay@Prashidikas-PC MINGW64 /d/Desktop/Projects/CompilerMate
$ python -m compilermate.main
=====
                CompilerMate
        Two-Way Developer & Compiler Interaction
=====
SOURCE > int x = 10
COMPILER: Error at line 1, column 11: expected ";"
YOU: > explain
COMPILER: Expected ';' after expression (expected ';', got 'EOF') at line 1, column 11.
YOU: > suggest fixes
COMPILER: [1] Insert missing semicolon at line 1, column 11.
YOU: > apply 1
COMPILER: Fix applied. Recompiling...
COMPILER: Compilation successful.
```

Figure B. Complete Review 1 interaction: diagnostic, explanation, repair suggestion, repair application, and recompilation.

Figure C:

```
Prashidika Upadhyay@Prashidikas-PC MINGW64 /d/Desktop/Projects/CompilerMate
$ python -m compilermate.main
=====
                CompilerMate
        Two-Way Developer & Compiler Interaction
=====
SOURCE > int x=10;
COMPILER: Compilation successful.
```

Figure C. Successful compilation of the corrected source.

# Who is the compiler talking to?

You write code.

The compiler analyzes it.

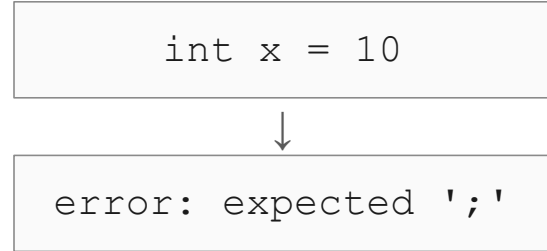
The compiler finds a problem.

The compiler tells you:

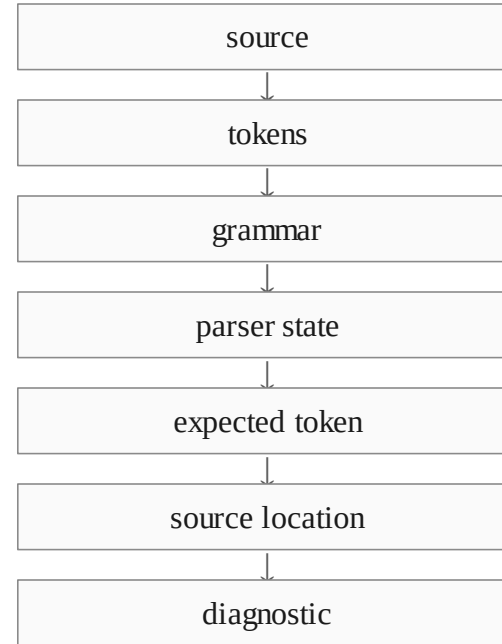
```
error: expected ';' 
```

*And that's where the conversation ends.*

## What I see



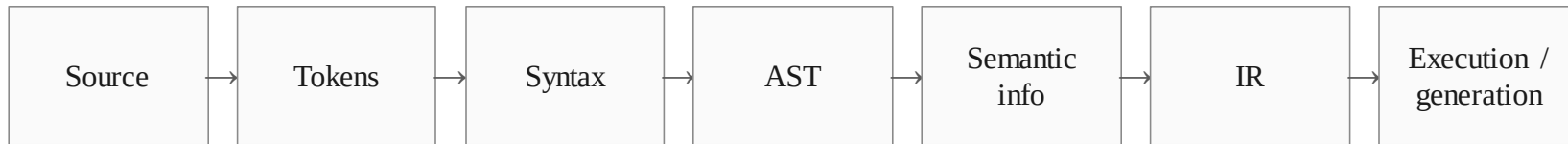
## vs. What the compiler is doing



*“It is the same event  
but the compiler has a richer representation of it than I get to interact with.”*



# Why does the compiler talk this way?



**The compiler already builds the context.**

**The developer usually receives the conclusion.**

*The audience already knows how a compiler is built. What matters here is what happens to that internal picture after it is built.*

## What the compiler represents

- ✓ source location
- ✓ expected token
- ✓ actual token / EOF
- ✓ grammar context
- ✓ compiler phase
- ✓ diagnostic information
- ✓ information needed for supported repairs

## What the developer can do after the diagnostic

read it

interpret it

edit

compile again

*The compiler has a state. The developer has a question.  
The interaction between them stops too early.*

# What if the compiler didn't have to stop at the error?

COMPILER: *"I found something."*

DEVELOPER: *"Why?"*

COMPILER: *"Here is why."*

DEVELOPER: *"What can I do?"*

COMPILER: *"Here are the supported options."*

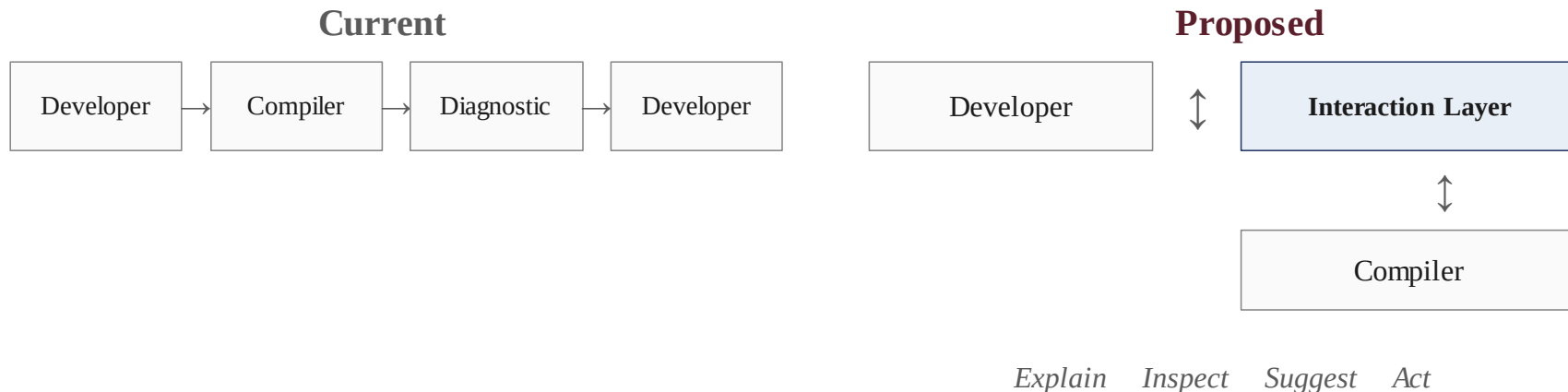
DEVELOPER: *"Do this one."*

COMPILER: *"Done. Let me check again."*

*What if a diagnostic was the beginning of a conversation, not the end of one?*

# CompilerMate

*A Two-Way Developer–Compiler Interaction Model*



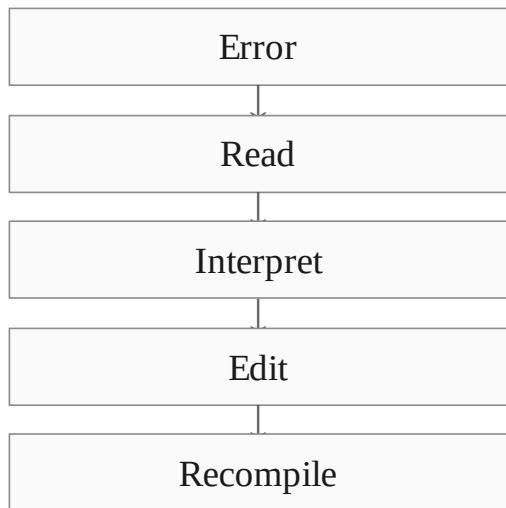
**The compiler pipeline remains.**

**The interaction model changes.**

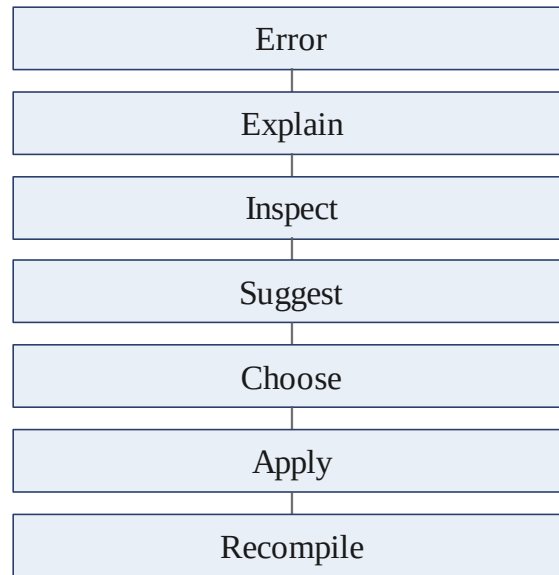
*CompilerMate treats the compiler's active analysis context  
as addressable session state.*

# What actually changes?

## Conventional



## CompilerMate

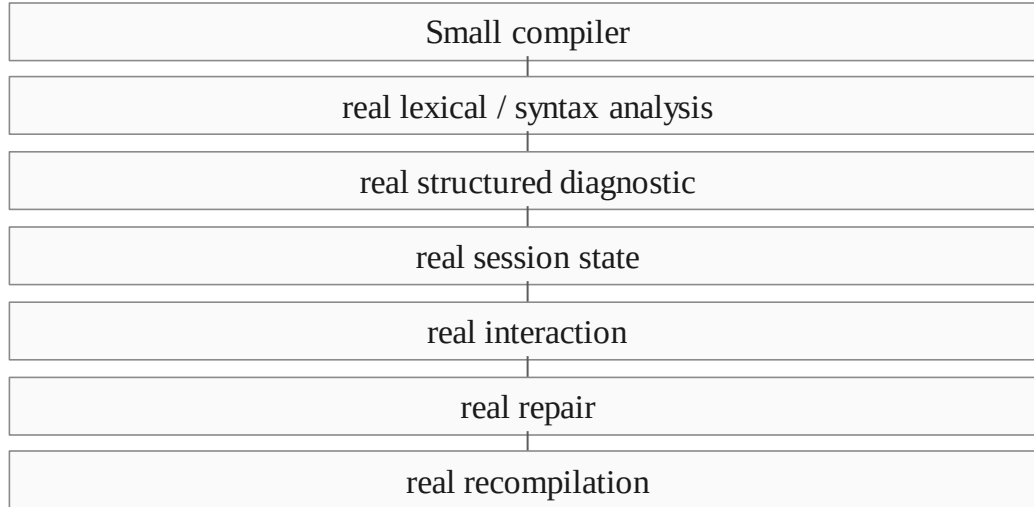


*The diagnostic becomes something you interact with —  
not just something you read.*

# A small world is enough.

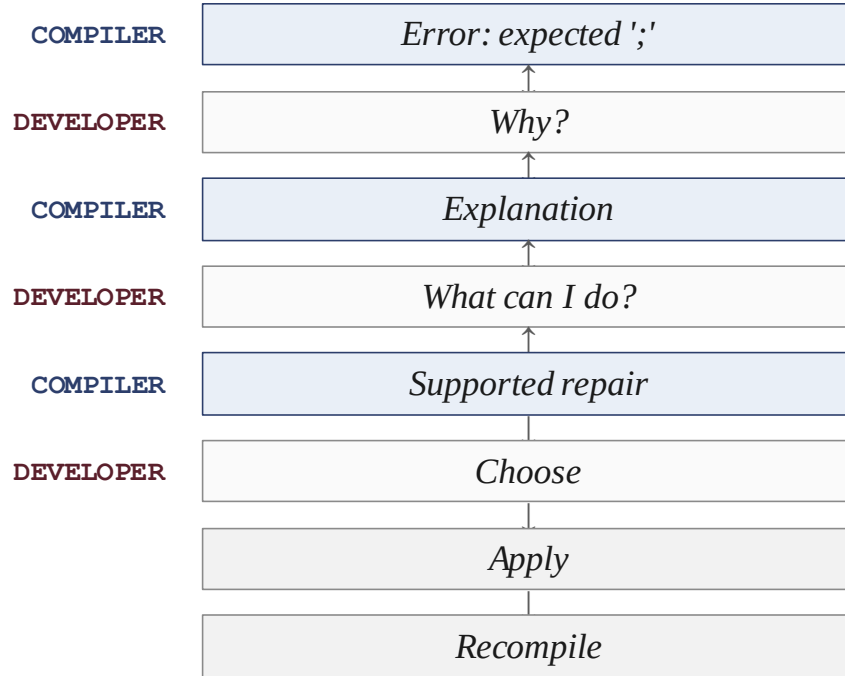
*I don't need to reproduce a production compiler.*

*I need enough machinery to create one real diagnostic and keep it addressable.*



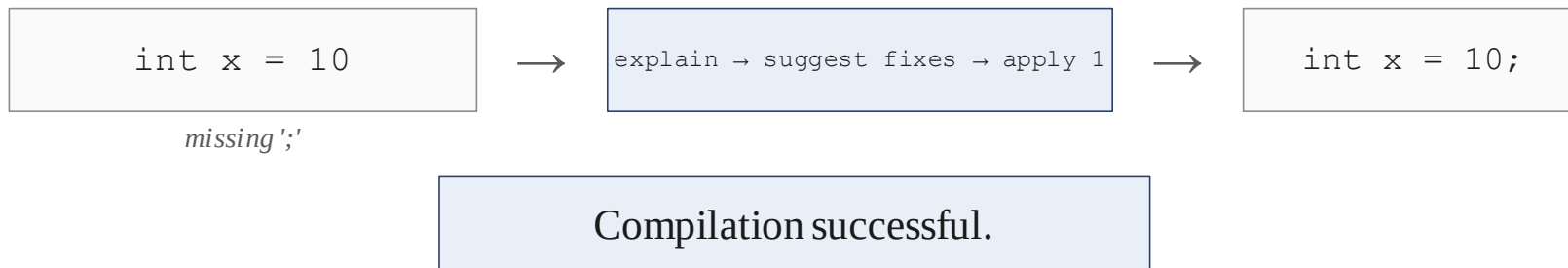
*The compiler is the experimental environment.  
The interaction model is the subject of the experiment.*

# What would this interaction actually look like?



*The diagnostic is now the object around which the interaction happens.*

# So what does the idea look like when it is made real?



```
Prashidika Upadhyay@Prashidikas-PC MINGW64 /d/Desktop/Projects/CompilerMate
$ python -m compilermate.main
=====
CompilerMate
Two-Way Developer✗Compiler Interaction
=====
SOURCE > int x = 10
COMPILER: Error at line 1, column 11: expected ";"
YOU: > explain
COMPILER: Expected ';' after expression (expected ';', got 'EOF') at line 1, column 11.
YOU: > suggest fixes
COMPILER: [1] Insert missing semicolon at line 1, column 11.
YOU: > apply 1
COMPILER: Fix applied. Recompiling...
COMPILER: Compilation successful.
```

```
Prashidika Upadhyay@Prashidikas-PC MINGW64 /d/Desktop/Projects/CompilerMate
$ python -m compilermate.main
=====
CompilerMate
Two-Way Developer✗Compiler Interaction
=====
SOURCE > int x=10;
COMPILER: Compilation successful.
```

**Implementation:** [github.com/prashidika111/compiler-mate](https://github.com/prashidika111/compiler-mate)



# What does this small experiment establish?

## The compiler can...

produce a real, structured diagnostic.

## The developer can...

continue interacting with that retained diagnostic.

## The system can...

apply a bounded repair and verify it through recompilation.

*A diagnostic doesn't have to be the end of the exchange.*

***It can be the thing the exchange is about.***

**The compiler already knows.**

*What if we could ask?*